Reinforcement Learning for Cybersecurity: A Flow-Based Defender for Binary Permit/Block Decisions

Javier Rivero Iglesias

May 2026

Contents

Chapter 1

State of the Art

1.1 Introduction and Scope of the Chapter

This chapter maps the technical background for a reinforcement-learning-based cybersecurity defender that operates on flow-level network observations. The scope is deliberately broader than a comparison of binary classifiers. The thesis combines network intrusion detection, flow-feature representation, public benchmark datasets, supervised machine learning baselines, a Gymnasium-style dataset-as-environment formulation, QRDQN, leakage-aware preprocessing, stricter validation checks, data-efficiency experiments, and planned or preferred validation on independently captured laboratory traffic.

The chapter therefore follows a funnel. It first reviews network intrusion detection and traffic representation, then situates CICIDS2017 among public datasets, then discusses supervised and deep learning baselines, and finally places the RL formulation within both reinforcement-learning theory and prior RL/DRL work for intrusion detection and cyber defense. The final sections focus on methodological risks, evaluation protocols, training-scale evidence, and the research gap addressed by this thesis.

The central claim is cautious. Reinforcement learning for intrusion detection already exists, and public NIDS benchmarks are widely used (Yang et al.

2026; Kheddar et al. 2025; Gueriani et al. 2024). This thesis does not claim to introduce the first RL IDS, to prove QRDQN as the NIDS state of the art, or to demonstrate production-ready inline prevention. It studies a reproducible, offline, flow-level `PERMIT/BLOCK` decision pipeline and evaluates how that formulation behaves under controlled internal benchmarks, baseline comparison, and progressively stricter validation.

The review also separates three levels that are often conflated. The first level is monitoring: collecting packets, flows, logs, or derived events and deciding whether they are suspicious. The second is prediction: training a model to map traffic representations to benign or malicious labels. The third is control: taking actions that change the future network state. This thesis works mainly at the first two levels. It uses the language of `PERMIT` and `BLOCK` because the environment exposes a binary security decision, but the current implementation is not an inline control plane. That distinction shapes every later claim.

1.2 Intrusion Detection and Network Security Monitoring

Intrusion Detection Systems monitor host or network activity for evidence of attacks, misuse, or policy violations (Scarfone et al. 2007). A Host Intrusion Detection System observes endpoints, operating-system events, logs, processes, file-integrity changes, or system calls. A Network Intrusion Detection System observes traffic crossing links, taps, mirrored switch ports, packet brokers, virtual-network interfaces, or flow collectors. The two perspectives are complementary: HIDS can see local process and file context that network sensors cannot, while NIDS can inspect lateral movement, scans, command-and-control traffic, and aggregate communication patterns without installing an agent on every host.

IDS must also be distinguished from Intrusion Prevention Systems. An IDS primarily detects and reports suspicious activity. Its output may be an alert, log event, enriched flow record, ticket, SIEM event, or forensic artifact. An IPS is positioned inline and can block, drop, modify, rate-limit, reset, or redirect traffic (Scarfone et al. 2007). In practice, products and architectures often blur this boundary because a detection engine may feed firewall rules, EDR actions, SOAR playbooks, or SDN policy updates. For evaluation, however, the distinction matters. A detector can tolerate a different false-positive profile from an inline blocker, because false alerts mainly affect analyst workload whereas false blocks can interrupt legitimate business traffic.

Classical IDS taxonomies distinguish signature-based, anomaly-based, specification-based, and hybrid approaches (Scarfone et al. 2007). Signature-based detection matches known malicious patterns: byte sequences, rule predicates, protocol conditions, command strings, exploit traces, or known behavior combinations. Its strength is precision and explainability for known threats. Its weakness is dependency on rule maintenance and poor coverage of novel attacks, polymorphic behavior, encrypted payloads, and environment-specific variations.

Anomaly-based detection models expected behavior and flags deviations (Sommer et al. 2010; Ring et al. 2019). This is the family most naturally associated with ML-based NIDS, but the word anomaly should not be interpreted as automatically meaning unsupervised learning. Many public NIDS studies are supervised classifiers trained on labeled benign and attack flows, even when the broader motivation is anomaly detection. The operational challenge is that benign traffic is broad and non-stationary, attack prevalence is low, labels are incomplete, and changes in business activity can look anomalous without being malicious.

Specification-based detection encodes allowed or expected behavior, for example protocol state machines, allowed command sequences, or restricted industrial-control workflows. It can be powerful when the monitored system

has narrow, stable behavior, but it requires domain knowledge and maintenance. Hybrid systems combine these ideas, such as signature rules for known exploits, statistical anomaly scores for unusual flow behavior, and host telemetry for endpoint confirmation. Modern monitoring programs rarely depend on one detection paradigm alone.

Alerting and active response form a separate axis. A system may only produce alerts for later triage, may enrich alerts with contextual evidence, may recommend response steps, or may trigger active containment. This thesis is intentionally on the conservative side of that axis. It studies an offline NIDS-like decision model over extracted flow rows. The model emits `PERMIT/BLOCK` predictions during experiments, but it does not drop packets, rewrite firewall policies, update SDN rules, quarantine hosts, or operate in a live feedback loop. The NIDS literature is therefore relevant, but production IPS claims are outside the supported scope.

1.3 Network Traffic Representation

Network traffic can be represented as packets, payload bytes, sessions, connections, or flows. Packet-level representations preserve per-packet timing, headers, sizes, directions, flags, and sometimes partial payload. They support fine-grained protocol analysis and sequence modeling, but they are expensive to store and process on high-speed networks. Payload-level representations retain application content and can identify exploits or malware signatures that are invisible in metadata. Their usefulness is limited by encryption, privacy constraints, legal restrictions, and CPU cost.

Session- or connection-level representations group packets belonging to the same exchange, often using the five-tuple of source IP, destination IP, source port, destination port, and protocol. They may include connection state, handshake behavior, application metadata, or TLS/HTTP-derived attributes.

Flow-level representations are more compact: they aggregate packets over a time window or connection-like record and summarize behavior through duration, bytes, packets, directionality, inter-arrival times, packet-length statistics, TCP flag counts, and active or idle periods (Sperotto et al. 2010; Umer et al. 2017).

NetFlow and IPFIX-style records are the operational reference point for scalable flow monitoring. Routers, switches, probes, and collectors export compact flow records that can be retained for long periods and processed at scale. These records are attractive for NIDS because they reduce volume and avoid payload inspection. They are also limited because deployments often export only a subset of possible fields, may sample traffic, and may omit richer statistics used in academic datasets. This creates a gap between production flow telemetry and feature-rich benchmark CSVs.

CICFlowMeter-style features sit on the richer, research-oriented side of that gap. CICIDS2017 provides PCAPs and generated bidirectional flow CSVs, with features such as flow duration, forward and backward packet and byte counts, inter-arrival-time statistics, packet-length statistics, TCP flag counts, header-length aggregates, flow rates, and active/idle timing (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). These features are convenient for tabular ML because each flow becomes a fixed-width row, but they depend on offline PCAP reconstruction and feature-extraction choices.

The suitability of flow-level representation is therefore conditional. It is suitable for this thesis because the implemented defender consumes fixed numerical vectors, because CICIDS2017 is already released in flow form, and because the planned Phase 2 path also extracts flow CSVs before inference. It is limited because payload semantics, exact packet ordering, application content, and some multi-flow temporal dependencies are lost. Attacks whose evidence is mainly content-level, host-level, or long-horizon behavioral may be only indirectly visible to a flow classifier (Sperotto et al. 2010; Umer et al.

2017).

Feature availability is another limitation. A feature computed by CICFlowMeter from full PCAPs may not be available from a production NetFlow/IPFIX exporter, or may be computed differently. Sarhan et al. address this problem by mapping heterogeneous NIDS feature sets toward a standard NetFlow-aligned representation to improve generalisability and explainability (Sarhan et al. 2021). This thesis does not adopt Sarhan's exact standard feature set, but it follows the same methodological motivation: define a stable feature contract, document what is present or missing, and avoid treating a benchmark-specific CSV schema as if it were universal.

The current project uses a canonical flow schema with 76 feature values and a 76-value missingness mask, producing 152-dimensional observations. The mask records whether each canonical value was present and valid, rather than silently hiding missing or imputed fields. This matters when moving from CICIDS2017 to laboratory traffic because different extraction pipelines may not produce exactly the same columns. The mask does not make missing data harmless, and it does not solve domain shift. It makes the model input auditable: a reader can distinguish a measured zero from an imputed placeholder.

Flow representation also changes the privacy and leakage profile. Removing payloads can reduce privacy exposure, but metadata can still identify users, services, schedules, and scenarios. IP addresses, absolute timestamps, Flow IDs, endpoint identity, capture-day structure, and ports used as scenario proxies can leak labels rather than represent attack behavior (Arp et al. 2020; Kaufman et al. 2011). The thesis therefore treats representation and feature selection as security-relevant design choices, not only preprocessing details.

1.4 Public Datasets for Network Intrusion Detection

Public NIDS datasets enable shared experimentation, reproducibility, and comparison across papers (Ring et al. 2019; Thakkar et al. 2020). They also constrain conclusions. A dataset reflects its traffic generator, topology, capture process, feature extractor, labels, attack scripts, and release format. Performance on a public benchmark is therefore evidence about a controlled task, not direct evidence of deployment readiness (Apruzzese et al. 2022; Layeghy et al. 2023).

KDDCup99 and NSL-KDD are historically important because they shaped early IDS evaluation (KDD Cup 1999; Tavallaee et al. 2009). KDDCup99 contains connection records derived from DARPA-era traffic and uses a 41-feature schema with attack categories such as DoS, Probe, R2L, and U2R. NSL-KDD reduced some redundancy problems and made the benchmark easier to use, but it retained the same basic origin and feature logic. These datasets are useful for understanding the history of IDS evaluation, but they are weak evidence for modern flow-based defense because their traffic, attack behavior, and schema are old (Tavallaee et al. 2009; Ring et al. 2019). In this repository, NSL-KDD is therefore historical context, not the final path for the current defender.

UNSW-NB15 is a more recent cyber-range dataset generated with contemporary benign and malicious traffic, raw captures, and engineered features (Moustafa and Slay 2015; Moustafa and Slay 2016). It remains useful because it broadened the field beyond KDD-style benchmarks and includes a richer set of attack categories. CICIDS2017 and CSE-CIC-IDS2018 are widely used CIC-family datasets for flow-based NIDS research, both providing modern lab-generated attack scenarios and flow features derived from packet captures (Sharafaldin et al. 2018; Communications Security Establishment et al. 2018).

Bot-IoT and ToN_IoT broaden the landscape toward IoT and IIoT traffic, where device behavior, telemetry, and attack patterns differ from enterprise networks (Koroniotis et al. 2019; Moustafa, Ahmed, et al. 2020; Alsaedi et al. 2020).

Dataset	Benchmark role	Representation	Main strengths	Main caveats
KDDCup99	Legacy historical benchmark	Connection records with 41 handcrafted features	Very widely used; simple baseline for older IDS literature (KDD Cup 1999)	Outdated traffic; redundancy; weak proxy for modern NIDS (Ring et al. 2019)
NSL-KDD	Cleaned legacy benchmark	Same KDD-style connection schema	Reduces some duplicate-record issues; useful for historical comparison (Tavallaee et al. 2009)	Still inherits old DARPA/KDD assumptions and feature design
UNSW-NB15	Modern lab/cyber-range dataset	PCAPs, logs, and engineered tabular features	More contemporary attack taxonomy and richer features (Moustafa and Slay 2015)	Controlled environment; not direct production evidence
CICIDS2017	Main internal thesis benchmark	PCAPs and CICFlowMeter flow CSVs	Public PCAPs; multiple attack days; rich flow features (Sharafaldin et al. 2018)	Known label, extraction, duplication, and split-sensitivity concerns (Engelen et al. 2021; Lanvin et al. 2023)
CSE-CIC-IDS2018	Larger CIC-family benchmark	PCAPs and CICFlowMeter-style flows	Broader CIC/CSE collaboration; useful modern comparison (Communications Security Establishment et al. 2018)	Similar lab-generation and flow-artifact caveats (L. Liu et al. 2022)
Bot-IoT	IoT/botnet benchmark	PCAPs, Argus-style flows, CSV exports	IoT-focused botnet, scan, DoS/DDoS, and exfiltration scenarios (Koroniotis et al. 2019)	Strong imbalance and dataset-specific topology artifacts
ToN_IoT	IoT/IIoT telemetry benchmark	Network flows, host logs, and sensor telemetry	Multi-modal IoT/IIoT perspective (Alsaedi et al. 2020)	Niche environment; identifier and scenario artifacts remain possible

Table 1.1: Compact comparison of public NIDS datasets relevant to the thesis.

Table 1.1 is not a ranking. It shows why dataset choice must be aligned with claim strength. Historical benchmarks help situate the field; modern lab datasets support controlled comparison; IoT datasets test different traffic assumptions; external laboratory or production-like traffic is needed for stronger generalisation evidence. None of these datasets should be treated as a complete proxy for all real networks. Their value is highest when used with transparent

preprocessing, strong baselines, and validation protocols that test distribution shift rather than only random matched-distribution performance.

1.5 CICIDS2017 as the Main Internal Benchmark

CICIDS2017 is the main internal benchmark in this thesis because it provides labeled benign and attack traffic, raw PCAPs, and flow CSVs generated with CICFlowMeter-style features (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). It was designed to improve on older IDS datasets by using contemporary traffic profiles, realistic benign activity profiles, and multiple attack scenarios (Sharafaldin et al. 2018; Sharafaldin et al. 2019). These properties make it suitable for controlled experimentation with flow-level NIDS models.

Its structure also makes it attractive for thesis work. The dataset spans multiple capture days and attack families, and it is available both as raw packet captures and as generated CSV files. The PCAPs are closer to the original measurement evidence. The CSVs are easier to use for ML because CICFlowMeter has already reconstructed bidirectional flows and computed statistical features. This convenience is also a risk: once researchers work only with summarized CSVs, they may stop checking how the flows were generated, how labels were assigned, and whether rows preserve independence across train and test partitions.

CICFlowMeter reconstructs flows from packet traces using endpoint keys, directionality, timeout logic, and feature computations. Small differences in extraction settings can change the resulting rows, durations, counts, and labels. Engelen et al. show that CICIDS2017 contains practical issues in flow reconstruction and dataset consistency (Engelen et al. 2021). Lanvin et al.

further report errors that affect detection performance and show that apparently small dataset corrections can produce significant metric differences (Lanvin et al. 2023; Lanvin et al. 2022). Liu et al. analyze error prevalence in CIC-IDS-2017 and CSE-CIC-IDS-2018, while Rosay et al. provide a broader analysis and reconstruction-oriented discussion (L. Liu et al. 2022; Rosay et al. 2022). Dube's critique is especially relevant to claim discipline because it questions common interpretations of high scores on summarized CIC-IDS 2017 data (Dube 2024).

These critiques do not make CICIDS2017 useless. They make irresponsible use easier to identify. Common misuse patterns include treating the pregenerated CSVs as unquestioned ground truth, using random row-wise splits across all days, keeping identifiers or timestamps that expose scenario context, reporting only accuracy, ignoring class imbalance, and failing to document whether the data are original, corrected, reconstructed, or curated. Corrected or reconstructed variants can be valuable, but they are not interchangeable with the original CSV release. A thesis must name which variant it uses and avoid mixing results from different variants as if they came from one dataset.

The current repository responds to these concerns through a curated tracked dataset copy, a separate untracked raw reference copy, and loader-level cleaning in `src/load_cicids2017.py`. The adapter performs numeric coercion, invalid-value handling, canonical mapping, and anti-leakage exclusion of fields that should not be model features. The code also fits scaling on the training split and applies the fitted transform to the test split, rather than fitting on all data. The canonical mapping in `src/canonical_schema.py` excludes IPs, timestamps, Flow IDs, and specific ports from the canonical features.

The evaluation code also supports random splits, CSV/day-style splits, and leave-one-exact-CSV-out validation. Random split performance remains useful for checking whether the model and implementation can learn the internal benchmark. Harder day/file splits, shuffled-label checks, leave-one-exact-CSV-

out validation, and external lab-flow inference test different failure modes. A high score on the easiest setting is therefore not sufficient for a deployment claim. In this thesis, CICIDS2017 should be read as the main internal benchmark, not as a substitute for independent traffic validation.

1.6 Supervised Machine Learning and Deep Learning for NIDS

Most flow-based NIDS work is naturally framed as supervised learning: each traffic record is mapped to a benign, malicious, or attack-family label (H. Liu et al. 2019; Ahmad et al. 2021). Classical supervised methods include decision trees, Random Forests, SVMs, k-NN, logistic regression, Naive Bayes, and gradient-boosted ensembles. These models differ in assumptions and operational cost. Logistic regression is simple and interpretable but limited for non-linear feature interactions. Naive Bayes is fast and useful as a weak baseline, but its independence assumptions are unrealistic for correlated flow statistics. k-NN can capture local neighborhoods but scales poorly and is sensitive to feature scaling. SVMs can be strong on smaller datasets but become expensive on very large flow tables and require careful scaling and kernel selection.

Tree-based models remain especially important for tabular NIDS. Decision trees and Random Forests handle non-linear interactions, heterogeneous feature scales, irrelevant variables, and mixed feature distributions well (H. Liu et al. 2019; Apruzzese et al. 2022). Gradient-boosted decision-tree families such as XGBoost, LightGBM, and CatBoost are often strong tabular learners, but this chapter does not need separate claims about each implementation unless the thesis evaluates them directly. The important methodological point is simpler: an RL-based flow classifier must be compared against competent tabular supervised baselines, not only against weak or outdated methods.

This matters directly for the thesis. If each observation is a labeled flow row and the reward is derived from classification correctness, a supervised classifier is the natural baseline. A QRDQN agent cannot be evaluated only against its own reward curve, training loss, or a majority-class baseline. It must be compared with strong supervised methods under the same feature schema, split protocol, preprocessing, and metrics. The current repository includes a Random Forest baseline protocol aligned with the QRDQN splits, while the maintained results snapshot still marks the latest full metrics as pending. Until those artifacts exist, the chapter should describe the baseline obligation rather than claim a completed comparison.

Deep learning expands the design space through MLPs, CNNs, RNNs, LSTMs, autoencoders, attention mechanisms, transformer-style models, and graph-based approaches (Xiao et al. 2021; Asadullah et al. 2023; Sayeeduddin et al. 2022; Nguyen et al. 2022). MLPs treat flow features as generic numerical vectors. CNNs are used when packet bytes, feature matrices, or local feature patterns are encoded in a structured form. RNNs and LSTMs target temporal sequences of packets, flows, or events. Autoencoders often support anomaly detection by learning compressed representations of benign behavior and flagging reconstruction errors. Transformer-style models and graph neural networks are newer directions for sequence and structural traffic representations.

These architectures can report very high benchmark scores, especially on public datasets, but surveys also identify recurring weaknesses: unclear preprocessing, favorable random splits, missing class-imbalance analysis, limited external validation, and insufficient reproducibility (H. Liu et al. 2019; Ring et al. 2019; Arp et al. 2020). A deep model can overfit benchmark artifacts as easily as a shallow model. The model family alone does not solve leakage, class imbalance, feature availability, or domain shift.

Feature standardisation is another important supervised-learning lesson. Different datasets and tools expose overlapping but non-identical feature sets, which makes cross-dataset comparison difficult. Sarhan et al. proposed a

NetFlow-aligned feature mapping to improve generalisability and explainability across NIDS datasets (Sarhan et al. 2021). This thesis does not adopt that exact standard, but it adopts the same motivation: a fixed canonical schema makes the model interface auditable and allows CICIDS2017 and laboratory flow inputs to be compared through one preprocessing contract. The missingness mask extends this idea by making absent features visible to the model and to the evaluator.

1.7 Reinforcement Learning Foundations

Reinforcement learning studies how an agent selects actions in an environment to maximize return (Sutton et al. 2018). Its core concepts are states or observations, actions, rewards, policies, value functions, and transitions. In a Markov decision process, the current state summarizes the information needed for future decision-making, and actions influence both immediate reward and later states (Sutton et al. 2018). This sequential property is what distinguishes RL from ordinary supervised classification.

A policy specifies how the agent chooses actions from observations. A value function estimates expected return from a state, while an action-value function estimates expected return after taking a particular action in a state. Rewards define the task objective, and return aggregates future rewards, often with discounting. These definitions matter because changing the reward function changes what the agent is optimized to do. In a security setting, a reward that penalizes false negatives more heavily than false positives encodes one operational preference; it is not a neutral fact about all networks.

The episodic versus continuing distinction is also relevant. Episodic tasks have boundaries and terminal states, such as a game episode or a simulated incident-response scenario. Continuing tasks model ongoing operation, such as continuous network monitoring. A static dataset-as-environment formulation

often imposes artificial episodes: an episode may be a fixed number of sampled rows, a pass through a dataset, or a file. That is acceptable for experimentation, but it is weaker than an environment where the defender's action changes the next state.

Classical Q-learning is a model-free, off-policy temporal-difference method that learns action values without requiring a model of environment dynamics (Watkins et al. 1992). Model-free methods learn directly from interaction or logged transitions, while model-based methods learn or use a transition model for planning. On-policy methods learn about the policy that is currently generating behavior; off-policy methods can learn about a target policy while behavior is generated differently. DQN and QRDQN inherit the value-based, off-policy framing. Experience replay, introduced earlier as a mechanism for reusing past experience, also became central to later deep Q-learning systems (L.-J. Lin 1992).

Tabular RL is not appropriate when observations are high-dimensional numerical vectors. A lookup table over all possible flow-feature combinations is infeasible, and small numerical changes could create effectively new states. Function approximation is therefore necessary. Deep RL uses neural networks to approximate policies or value functions over large observation spaces. In this thesis, the observation is a 152-dimensional vector: 76 canonical flow values plus 76 missingness-mask values. That justifies a DQN-family algorithm more than a tabular RL algorithm, but it does not by itself justify RL over supervised learning.

Cybersecurity can contain genuine sequential RL problems. A defender may isolate a host, patch a service, change routing, deploy a countermeasure, or choose where to inspect next, and those actions can affect later observations and attacker opportunities. However, a static labeled-flow dataset is a weaker setting. If one flow row is sampled, classified, and followed by another row that is not causally affected by the previous action, the problem resembles cost-sensitive classification or a contextual decision process more than full au-

tonomous cyber defense (Levine et al. 2020; Fujimoto et al. 2019). The thesis must keep this distinction explicit.

1.8 Deep Q-Learning and Distributional Reinforcement Learning

Deep Q-Learning replaces a tabular action-value function with a neural network, making value-based RL applicable to high-dimensional observations (Mnih et al. 2015). DQN popularized two stabilizing mechanisms: replay memory and a separate target network (Mnih et al. 2015). Replay memory breaks short-term correlations by sampling past transitions for training and improves data reuse. A target network stabilizes bootstrapped targets by keeping the target calculation separate from rapidly changing online network weights.

Later DQN variants addressed known weaknesses. Double DQN reduces overestimation bias by decoupling action selection from target evaluation (Hasselt et al. 2016). Dueling DQN separates state value and action advantage estimation, which can help when many actions have similar values in a state (Wang et al. 2016). Prioritized Experience Replay samples transitions according to learning signal rather than uniformly (Schaul et al. 2016). Rainbow combined several improvements into a single DQN-family agent, including double Q-learning, dueling networks, prioritized replay, multi-step returns, distributional learning, and noisy exploration (Hessel et al. 2018).

Distributional reinforcement learning changes the object being estimated. Instead of only estimating expected return, it approximates the distribution of possible returns (Bellemare et al. 2017). C51 represents the return distribution over a fixed categorical support. QRDQN uses quantile regression to approximate return quantiles, avoiding the same fixed support assumption (Dabney

et al. 2018). QRDQN is therefore not a separate paradigm from DQN; it is a distributional, value-based member of the DQN family.

The security motivation is intuitive but must be stated carefully. False positives and false negatives have asymmetric consequences, and rare high-cost errors can matter more than a mean score suggests. A distributional value estimate may be useful for studying return structure under asymmetric rewards. It may expose whether a decision has a narrow or broad return distribution under the learned model. However, distributional RL does not automatically provide calibrated uncertainty, operational safety, risk-sensitive behavior, or superiority over scalar DQN. Those properties would require explicit decision rules, calibration, tail-risk objectives, or evaluation criteria beyond simply training a distributional agent (Bellemare et al. 2017; Dabney et al. 2018).

In this thesis, QRDQN is an exploratory algorithmic choice. It is suitable for a discrete binary action space and moderately high-dimensional flow observations, and it is available through the SB3-Contrib implementation used by the project (Stable-Baselines3 Contributors 2023; Farama Foundation 2023). The algorithmic choice is coherent: two actions, numerical observations, off-policy value learning, replay, and a distributional target. The evidential claim remains empirical. QRDQN should not be described as proven superior for NIDS unless same-protocol comparisons against DQN, Random Forest, and other baselines support that claim.

1.9 Reinforcement Learning for Intrusion Detection and Cyber Defense

RL and DRL for intrusion detection already form an active literature (Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024; Adawadkar et al. 2022). Prior work includes DQN-style intrusion classifiers, actor-critic approaches,

adversarial-environment formulations, feature-selection agents, SDN or IoT-focused systems, and autonomous cyber-defense simulations. The field is heterogeneous, so papers should not be treated as directly comparable unless their task formulation, datasets, actions, rewards, and validation protocols are aligned.

One branch is close to this thesis: dataset-as-environment intrusion detection. Lopez-Martin et al. reformulated supervised intrusion detection as a deep RL problem by treating labeled records as states, class predictions as actions, and reward as a function of correctness (Lopez-Martin, Carro, et al. 2020). Their later work extended a related offline RL formulation across several datasets, including CICIDS2017 and UNSW-NB15 (Lopez-Martin, Sanchez-Esguevillas, et al. 2021). Ren et al. used DRL for feature selection and classification on CSE-CIC-IDS2018 (Ren et al. 2022). These works show that RL-style formulations for IDS are precedented.

Other IDS-oriented RL work explores DQN, Double DQN, adversarial-environment DQN, actor-critic, SAC, DDPG, and Rainbow-style designs (Caminero et al. 2019; Han et al. 2020; Hsu et al. 2023; Alavizadeh et al. 2021; Hossain et al. 2025). The details vary widely. Some papers use binary classification, others multi-class attack labels, feature selection, threshold control, or SDN-specific actions. Many still evaluate on static public datasets. That makes them relevant background for the thesis, but it also limits direct comparability. A paper using NSL-KDD with random splits and correctness rewards is not evidence that an offline QRDQN defender will generalize from CICIDS2017 to laboratory traffic.

A second branch studies more genuinely sequential cyber defense. POMDP and attack-graph approaches model defensive actions under uncertainty (Hu et al. 2020). CybORG provides a gym-style environment for autonomous cyber agents (Standen et al. 2021). Broader autonomous cyber defense work discusses agents that select countermeasures in simulated or emulated networks (Palmer et al. 2023; Center for Security and Emerging Technology et al. 2023).

These settings are conceptually different from this thesis because defender actions can alter future state. They represent a stronger form of RL problem than static row classification.

The thesis therefore belongs primarily to the first branch: offline flow-level detection formulated through an RL interface. It can reference autonomous cyber defense as broader context and future direction, but it should not imply that the current system performs multi-agent, adversarial, or live network-control learning. The cleanest wording is that the thesis implements a dataset-as-environment NIDS benchmark with a cost-sensitive binary action interface, not a full cyber-range defender.

1.10 Classification-as-RL and Binary Security Decisions

The `PERMIT/BLOCK` formulation belongs in the classification-as-RL discussion. In the current environment, each flow observation is presented to the agent, the action is binary, and reward is computed from the action and ground-truth label. `PERMIT` corresponds to accepting traffic as benign, while `BLOCK` corresponds to identifying it as attack. A false negative permits an attack flow; a false positive blocks benign traffic.

This framing has a real methodological benefit: it makes cost-sensitive security decisions explicit. IDS errors are not symmetric in practice. Missing an attack and blocking benign traffic have different operational meanings, and the preferred trade-off depends on the defended environment (Lee et al. 2002; Cárdenas et al. 2006). Reward design can encode this asymmetry directly, which is useful for a thesis studying defensive decision rules rather than only label prediction.

The limitation is equally important. If the dataset does not react to the

action, then `PERMIT/BLOCK` is not an active control action. It is an offline decision label over a recorded or extracted flow row. The next row does not change because the agent blocked or permitted the previous row. The formulation is therefore closer to reward-engineered classification than to a firewall, IPS, SDN controller, or autonomous cyber-defense agent. This is not a flaw if stated clearly. It becomes a flaw only if the chapter overclaims live blocking, deployment readiness, or fully sequential control.

The reward function also needs careful interpretation. A positive reward for a true positive and a larger penalty for a false negative can express the idea that missed attacks are costly. A false-positive penalty can express the cost of disrupting benign traffic or overwhelming analysts. A reward of zero or small value for true negatives can keep the focus on attack detection. These are design choices, not universal security economics. Different organizations would choose different costs depending on business continuity, threat model, analyst capacity, and tolerance for missed attacks.

Because the formulation is close to cost-sensitive classification, supervised baselines are mandatory. A strong Random Forest, and ideally other tabular baselines, help determine whether QRDQN adds empirical value beyond the canonical features, preprocessing, and reward design. The same metrics must be computed from both RL and supervised outputs. Training reward alone is not enough, because a reward curve can improve while precision, recall, or false-positive behavior remains operationally poor. The thesis should interpret any RL advantage as benchmark- and protocol-specific unless broader evidence is available.

1.11 Methodological Pitfalls in ML/DL/RL-Based NIDS

The NIDS literature contains repeated warnings against overinterpreting benchmark accuracy. Sommer and Paxson argued that ML-based intrusion detection is difficult to validate in realistic environments because benign traffic is diverse, attacks are rare, labels are hard to obtain, and operational base rates differ sharply from benchmark conditions (Sommer et al. 2010). Arp et al. identify similar risks across machine learning in computer security, including data leakage, sampling bias, inappropriate metrics, and weak experimental design (Arp et al. 2020).

Leakage is broader than accidentally placing the same row in train and test. It includes any target-related information that would not legitimately be available at prediction time (Kaufman et al. 2011). In NIDS, leakage can enter through endpoint identifiers, absolute timestamps, Flow IDs, scenario-specific ports, global preprocessing fitted before splitting, duplicate or near-duplicate flows, and train/test partitions that share capture context. A model that learns that a particular IP, port, day, or Flow ID corresponds to an attack may score well without learning portable malicious behavior.

Temporal and scenario leakage are especially important. Public NIDS datasets are often organized by day, capture file, attack window, or scripted scenario. A random row-wise split can place highly related traffic on both sides of the train/test boundary. Even when rows are not exact duplicates, they may share hosts, attack tools, timing regimes, or extraction artifacts. This is especially important for CICIDS2017 because later studies report errors or artifacts in flow extraction, labeling, and summarized CSV use (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022; Rosay et al. 2022; Dube 2024).

Preprocessing can also leak. Scaling, imputation, feature selection, outlier clipping, or dimensionality reduction must be fitted on the training partition

and then applied to validation or test data. If a scaler is fitted on the full dataset before splitting, distributional information from the test set has entered training. This may look harmless compared with label leakage, but in security datasets with strong day or scenario structure it can still inflate confidence. The repository's use of train-fitted `StandardScaler` objects is therefore part of the methodological design, not only a coding convenience.

Class imbalance and the base-rate problem complicate interpretation. In operational networks, attacks are typically rare relative to benign traffic. A detector can achieve high accuracy while missing many attacks, or can achieve high recall while producing too many false alerts to investigate. Axelsson's base-rate argument remains relevant because even low false-positive rates can dominate alert streams when true attack prevalence is low (Axelsson 2000). This is why accuracy should not be the headline measure for NIDS.

RL introduces additional pitfalls. Deep RL training can be stochastic and sensitive to random seeds, hyperparameters, reward shaping, replay-buffer settings, exploration, and environment details. In a static dataset-as-environment setting, strong results may reflect the same feature and split advantages that benefit supervised models. Therefore, any claim that QRDQN is better than a supervised baseline must be supported by same-protocol comparison and should acknowledge run-to-run variance when repeated seeds are not available. The chapter should treat single-run gains as preliminary unless backed by repeated experiments or strong validation artifacts.

1.12 Evaluation Protocols, Metrics, and Reproducibility

Evaluation should match the strength of the claim. Same-dataset random splits are acceptable for internal learning checks, but they are weak evidence

for deployment. A practical validation ladder for this thesis starts with random stratified splits, then moves to temporal or CSV/day splits, leave-one-scenario or leave-one-exact-CSV-out validation, cross-dataset testing with a harmonized feature schema, and finally independently captured laboratory or production-like traffic. Each step increases distributional separation between training and testing data, and therefore supports stronger generalisation claims.

The current repository reflects this ladder. Random split training verifies that the QRDQN implementation, feature schema, reward function, and training loop can learn the controlled benchmark. Check B with shuffled labels tests whether performance collapses toward a majority-class baseline when labels no longer contain real signal. Check C uses a harder CSV/day split. The leave-one-exact-CSV-out script tests whether holding out one original CICIDS2017 CSV changes behavior. Phase 2 offline inference on laboratory flow CSVs tests the separate question of whether a model trained on the benchmark can process independently extracted flows.

Metrics must also be interpreted operationally. Accuracy is useful but insufficient for imbalanced NIDS. Precision indicates how trustworthy positive alerts are. Recall indicates how much attack traffic is detected. F1 provides a compact balance, but it hides the actual false-positive and false-negative trade-off. False-positive rate maps to benign traffic incorrectly blocked or alerted. False-negative rate maps to malicious traffic missed by the detector (Axelsson 2000; Fawcett 2006; Saito et al. 2015). Confusion matrices, per-class precision, recall, F1, TP, FP, FN, TN, FPR, and FNR should be preferred over a single headline number (Milenkoski et al. 2015).

Cost-sensitive evaluation is relevant because the correct operating point depends on context. NIST guidance and IDS evaluation literature both emphasize trade-offs between false positives and false negatives (Scarfone et al. 2007; Cárdenas et al. 2006). In this thesis, the asymmetric reward design should be treated as a scenario assumption and evaluated empirically, not as a universal cost model for all networks. If the reward penalizes false negatives

heavily, the resulting policy may prefer blocking more flows. That behavior must be judged through precision, recall, false-positive rate, and the specific cost assumptions stated in the experiment.

Reproducibility is part of the evidence, not an administrative detail. Cybersecurity ML results should preserve dataset version, exact dataset variant, feature schema, excluded fields, preprocessing order, split logic, scaler state, random seeds, model configuration, reward configuration, run ID, metrics, and predictions where feasible. Olszewski et al. show that reproducibility remains a major problem in ML security research (Olszewski et al. 2023). The repository's use of saved configs, metrics, model artifacts, validation outputs, and run folders is therefore a methodological strength, but missing artifacts should remain clearly labeled as pending.

This reporting discipline also protects against accidental overclaiming. A result tied to the C03 full random-split QRDQN run is a result for that run, with that split, reward configuration, preprocessing path, and artifact state. It should not silently become a claim about all QRDQN configurations, all CICIDS2017 variants, or all network traffic. The stronger the claim, the more complete the artifact trail must be.

1.13 Data Efficiency and Training-Scale Evaluation

Data efficiency asks how much labeled traffic is needed to reach useful performance, and how the learning curve changes as the training set grows. This question is important for NIDS because labeled attack traffic is expensive, traffic distributions change, and public benchmark abundance does not imply deployment data abundance (Ring et al. 2019; Apruzzese et al. 2022). It is also important for RL because deep RL methods can be sample-hungry, especially

when reward signals are sparse, noisy, delayed, or highly shaped (Sutton et al. 2018; Hessel et al. 2018).

Existing RL-IDS literature often trains on full public datasets and reports final benchmark scores, while controlled training-scale ablations are less common (Yang et al. 2026; Kheddar et al. 2025). Supervised few-shot and class-incremental NIDS work addresses related questions from a different angle (Di Monda et al. 2024), but the thesis question is narrower: under one feature schema and evaluation protocol, how does QRDQN scale with available training rows, and how does that compare with supervised baselines?

The answer matters because model choice and data need are connected. If a Random Forest reaches stable performance with far fewer rows than QRDQN under the same split, that is important evidence even if QRDQN later catches up. If QRDQN is more sensitive to reduced data or to attack-class imbalance, the thesis should report that as a limitation. If QRDQN is competitive at smaller budgets, the evidence should still be tied to the benchmark and validation protocol, not generalized to all NIDS settings.

The thesis positions training-scale experiments as methodological evidence. Budgets such as 100k, 250k, 500k, 1M, and 2M samples can test whether the RL formulation requires substantially more data than a supervised baseline to reach comparable performance. Such experiments should not be framed as proof of few-shot learning or deployment readiness. They are internal evidence about the data requirements of this particular formulation. The useful output is a learning-curve comparison with consistent preprocessing, random seeds where feasible, shared metrics, and explicit artifact paths.

1.14 Research Gap and Positioning of This Thesis

The literature supports a narrow but meaningful research gap. NIDS and flow-based ML are mature areas; public datasets such as CICIDS2017 are useful but fragile; supervised tabular baselines are strong; RL and DRL for IDS already exist; and autonomous cyber defense is a broader, more sequential field than the current implementation (Ring et al. 2019; H. Liu et al. 2019; Yang et al. 2026; Kheddar et al. 2025). The gap is not the absence of RL for intrusion detection.

The contribution of this thesis is a reproducible experimental study of an offline, flow-level, binary security-decision formulation. The system maps CICIDS2017 and later lab-flow inputs into a fixed 76-feature canonical schema, augments observations with a missingness mask to produce 152-dimensional inputs, exposes rows through a Gymnasium-compatible environment, and trains a QRDQN agent to choose between `PERMIT` and `BLOCK`. This makes the cost-sensitive decision interface explicit while preserving a clear comparison obligation against supervised baselines.

The thesis is also positioned methodologically. It treats CICIDS2017 as the main internal benchmark, not as proof of production readiness. It separates random-split results from stricter validation. It uses anti-leakage preprocessing and validation checks to reduce obvious artifact learning. It treats external lab-captured traffic as the preferred next evidential step, while acknowledging that current Phase 2 inference is offline and does not perform active blocking.

The final gap is therefore an integration gap: a leakage-aware, reproducible, cost-sensitive QRDQN benchmark for flow-level NIDS, evaluated against supervised baselines and interpreted through a validation ladder. The thesis can contribute by making the feature contract explicit, saving run artifacts, documenting reward assumptions, comparing against Random Forest under

matched splits, and showing how internal CICIDS2017 performance changes under stricter validation and training-scale constraints.

The defensible final claim is not that QRDQN is universally better than supervised NIDS models, nor that the project implements autonomous cyber defense. The defensible claim is that the thesis evaluates a QRDQN-based, cost-sensitive, flow-level defender under a reproducible benchmark and validation pipeline, and uses supervised baselines, leakage-aware checks, training-scale analysis, and lab-traffic inference to determine how far that formulation can be trusted. If later experiments show weak cross-file or lab-traffic performance, that does not invalidate the thesis. It sharpens its contribution by demonstrating the limits of benchmark-trained RL under more realistic distribution shift.

Chapter 2

Bibliography

[1] Amrin Maria Khan Adawadkar and Nilima Kulkarni. "Cyber-Security and Reinforcement Learning: A Brief Survey". In: *Engineering Applications of Artificial Intelligence* 114 (2022), p. 105116. DOI: 10.1016/j.engappai.2022.105116 (cit. on p. 16).

[2] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah, and Farhan Ahmad. "Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches". In: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: 10.1002/ett.4150 (cit. on p. 11).

[3] Hooman Alavizadeh, Julian Jang-Jaccard, and Hootan Alavizadeh. "Deep Q-Learning Based Reinforcement Learning Approach for Network Intrusion Detection". In: (2021). arXiv: 2111.13978 [cs.CR] (cit. on p. 17).

[4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems". In: *IEEE Access* 8 (2020), pp. 165130–165150. DOI: 10.1109/ACCESS.2020.3022862 (cit. on p. 8).

[5] Giovanni Apruzzese, Luca Pajola, and Mauro Conti. "The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems". In: *IEEE Transactions on Network and Service Management* 19.4 (2022), pp. 5152–5169. DOI: 10.1109/TNSM.2022.3157344 (cit. on pp. 7, 11, 23).

[6] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. "Dos and Don'ts of Machine Learning in Computer Security". In: (2020). arXiv: 2010.09470 [cs.CR] (cit. on pp. 6, 12, 20).

[7] Momand Asadullah, Jan Sana Ullah, and Naeem Ramzan. "A Systematic and Comprehensive Survey of Recent Advances in Intrusion Detection Systems Using Machine Learning: Deep Learning, Datasets, and Attack Taxonomy". In: *Expert Systems* (2023). DOI: 10.1155/2023/6048087 (cit. on p. 12).

[8] Stefan Axelsson. "The Base-Rate Fallacy and the Difficulty of Intrusion Detection". In: *ACM Transactions on Information and System Security* 3.3 (2000), pp. 186–205. DOI: 10.1145/357802.357804 (cit. on pp. 21, 22).

[9] Marc G. Bellemare, Will Dabney, and Rémi Munos. "A Distributional Perspective on Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, pp. 449–458. URL: https://proceedings.mlr.press/v70/bellemare17a.html (cit. on pp. 15, 16).

[10] Guillermo Caminero, Manuel Lopez-Martin, and Belen Carro. "Adversarial Environment Reinforcement Learning Algorithm for Intrusion Detection". In: *2019 International Joint Conference on Neural Networks (IJCNN)*. 2019, pp. 1–8. DOI: 10.1109/IJCNN.2019.8852380 (cit. on p. 17).

[11] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. 2017. URL: https://www.unb.ca/cic/datasets/ids-2017.html (cit. on pp. 5, 9).

[12] Alvaro A. Cárdenas, John S. Baras, and Karl Seamon. "A Framework for the Evaluation of Intrusion Detection Systems". In: *2006 IEEE Symposium on Security and Privacy (S&P)*. 2006, pp. 63–77. DOI: 10.1109/SP.2006.2 (cit. on pp. 18, 22).

[13] Center for Security and Emerging Technology and The Alan Turing Institute. *Autonomous Cyber Defense*. https://cset.georgetown.edu/publication/autonomous-cyber-defense/. 2023 (cit. on p. 17).

[14] Communications Security Establishment and Canadian Institute for Cybersecurity. *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. 2018. URL: https://registry.opendata.aws/cse-cic-ids2018/ (cit. on pp. 7, 8).

[15] Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. "Distributional Reinforcement Learning with Quantile Regression". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, pp. 2892–2901. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11791 (cit. on pp. 15, 16).

[16] Davide Di Monda et al. "Few-Shot Class-Incremental Learning for Network Intrusion Detection Systems". In: (2024). Exact venue and DOI require verification; see arXiv preprint search for Di Monda 2024 (cit. on p. 24).

[17] Rohit Dube. "Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research". In: *Journal of Computer Virology and Hacking Techniques* 20.1 (2024), pp. 203–211. DOI: 10.1007/s11416-023-00509-7 (cit. on pp. 10, 20).

[18] Gints Engelen, Vera Rimmer, and Wouter Joosen. “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”. In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009 (cit. on pp. 8, 9, 20).

[19] Farama Foundation. *Gymnasium Documentation*. Software documentation; access date 2026. 2023. URL: https://gymnasium.farama.org/ (cit. on p. 16).

[20] Tom Fawcett. “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010 (cit. on p. 22).

[21] Scott Fujimoto, David Meger, and Doina Precup. “Off-Policy Deep Reinforcement Learning without Exploration”. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, pp. 2052–2062. URL: https://proceedings.mlr.press/v97/fujimoto19a.html (cit. on p. 15).

[22] Afrah Gueriani, Hamza Kheddar, and Ahmed Cherif Mazari. “Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey”. In: (2024). Preprint; published version in IC2EM proceedings 2023. arXiv: 2405.20038 [cs.CR] (cit. on pp. 2, 16).

[23] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. “Characterization of Tor Traffic Using Time Based Features”. In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, pp. 253–262. DOI: 10.5220/0006105602530262 (cit. on pp. 5, 9).

[24] Dongkun Han, Zhiping Wang, Yihua Zhong, Wenhan Chen, Jiaze Yang, Shanqing Lu, Xinyan Shi, and Xia Yin. “Evaluating Network Intrusion Detection Systems for Different Vulnerability Conditions Using an Adversarial Environment DQN”. In: *IEEE Access* 9 (2020). IEEE Xplore

document ID 9289884; full author list requires verification, pp. 4345–4359. DOI: 10.1109/ACCESS.2020.3047430 (cit. on p. 17).

[25] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-Learning". In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. 2016, pp. 2094–2100. arXiv: 1509.06461 (cit. on p. 15).

[26] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, pp. 3215–3222. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11796 (cit. on pp. 15, 24).

[27] Md. Alamgir Hossain et al. "Deep Q-Learning Intrusion Detection System (DQ-IDS): A Novel Reinforcement Learning Approach for Adaptive and Self-Learning Cybersecurity". In: *ICT Express* (2025). Full author list requires verification; accessed via ScienceDirect. DOI: 10.1016/j.icte.2025.02.006 (cit. on p. 17).

[28] Chia-Wei Hsu, Chia-Hsuan Lin, et al. "A Soft Actor-Critic Reinforcement Learning Algorithm for Network Intrusion Detection". In: *Computers & Security* 136 (2023). Full author list requires verification against published paper, p. 103580. DOI: 10.1016/j.cose.2023.103580 (cit. on p. 17).

[29] Zhisheng Hu, Minghui Zhu, and Peng Liu. "Adaptive Cyber Defense Against Multi-Stage Attacks Using Learning-Based POMDP". In: *ACM Transactions on Privacy and Security* 24.1 (2020), pp. 1–31. DOI: 10.1145/3418897 (cit. on p. 17).

[30] Shachar Kaufman, Saharon Rosset, and Claudia Perlich. “Leakage in Data Mining: Formulation, Detection, and Avoidance”. In: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, pp. 556–563. DOI: 10.1145/2020408.2020496 (cit. on pp. 6, 20).

[31] KDD Cup. *KDD Cup 1999 Data*. Dataset page. 1999. URL: http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html (cit. on pp. 7, 8).

[32] Hamza Kheddar, Diana W. Dawoud, Ali Ismail Awad, Yassine Himeur, and Muhammad Khurram Khan. “Reinforcement-Learning-Based Intrusion Detection in Communication Networks: A Review”. In: *IEEE Communications Surveys & Tutorials* 27.4 (2025), pp. 2420–2469. DOI: 10.1109/COMST.2024.3484491 (cit. on pp. 2, 16, 24, 25).

[33] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. “Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset”. In: *Future Generation Computer Systems* 100 (2019), pp. 779–796. DOI: 10.1016/j.future.2019.05.041 (cit. on p. 8).

[34] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes”. In: *Risks and Security of Internet and Systems*. Vol. 13857. Lecture Notes in Computer Science. Springer, 2023, pp. 19–34. DOI: 10.1007/978-3-031-31108-6_2 (cit. on pp. 8, 10, 20).

[35] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. “Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research”. In: *Risks and Security of Internet and Systems*. Vol. 13561. Lecture Notes in Computer Science. Earlier presentation of dataset-critique work; see also Lanvin2023Errors. Springer, 2022 (cit. on p. 10).

[36] Siamak Layeghy and Marius Portmann. “Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems”. In: *Computers and Electrical Engineering* 108 (2023), p. 108692. DOI: 10.1016/j.compeleceng.2023.108692 (cit. on p. 7).

[37] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo, and Erez Zadok. “Toward Cost-Sensitive Modeling for Intrusion Detection and Response”. In: *Journal of Computer Security* 10.1–2 (2002), pp. 5–22. DOI: 10.3233/JCS-2002-101-202 (cit. on p. 18).

[38] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. “Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems”. In: *arXiv preprint arXiv:2005.01643* (2020). Preprint. arXiv: 2005.01643 [cs.LG] (cit. on p. 15).

[39] Long-Ji Lin. “Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching”. In: *Machine Learning* 8.3–4 (1992), pp. 293–321. DOI: 10.1007/BF00992699 (cit. on p. 14).

[40] Hongyu Liu and Bo Lang. “Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey”. In: *Applied Sciences* 9.20 (2019), p. 4396. DOI: 10.3390/app9204396 (cit. on pp. 11, 12, 25).

[41] Lisa Liu, Gints Engelen, Timothy M. Lynar, Daryl Essam, and Wouter Joosen. “Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018”. In: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, pp. 254–262. DOI: 10.1109/CNS56114.2022.9947235 (cit. on pp. 8, 10, 20).

[42] Manuel Lopez-Martin, Belén Carro, and Antonio Sanchez-Esguevillas. “Application of Deep Reinforcement Learning to Intrusion Detection for Supervised Problems”. In: *Expert Systems with Applications* 141 (2020), p. 112963. DOI: 10.1016/j.eswa.2019.112963 (cit. on p. 17).

[43] Manuel Lopez-Martin, Antonio Sanchez-Esguevillas, Juan I. Arribas, and Belén Carro. "Network Intrusion Detection Based on Extended RBF Neural Network With Offline Reinforcement Learning". In: *IEEE Access* 9 (2021), pp. 153153–153170. DOI: 10.1109/ACCESS.2021.3127689 (cit. on p. 17).

[44] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer, and Bryan D. Payne. "Evaluating Computer Intrusion Detection Systems: A Survey of Common Practices". In: *ACM Computing Surveys* 48.1 (2015), 12:1–12:41. DOI: 10.1145/2808691 (cit. on p. 22).

[45] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236 (cit. on p. 15).

[46] Nour Moustafa, Mohiuddin Ahmed, and Sherif Ahmed. "Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets". In: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, pp. 727–735. DOI: 10.1109/TrustCom50675.2020.00080 (cit. on p. 8).

[47] Nour Moustafa and Jill Slay. "The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set". In: *Information Security Journal: A Global Perspective* 25.1-3 (2016), pp. 18–31. DOI: 10.1080/19393555.2015.1125974 (cit. on p. 7).

[48] Nour Moustafa and Jill Slay. "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)". In: *2015 Military Communications and Information Systems Con-*

ference (MilCIS). 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942 (cit. on pp. 7, 8).

[49] Loc Gia Nguyen and Kohei Watabe. "Flow-Based Network Intrusion Detection Based on BERT Masked Language Model". In: *CoNEXT 2022 Student Workshop*. 2022. DOI: 10.1145/3565477.3569152. arXiv: 2306.04920 (cit. on p. 12).

[50] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler, and Patrick Traynor. ""Get in Researchers; We're Measuring Reproducibility": A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences". In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3433–3459. DOI: 10.1145/3576915.3623130 (cit. on p. 23).

[51] Gregory Palmer, Chris Parry, Daniel J. B. Harrold, and Chris Willis. "Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey". In: (2023). arXiv: 2310.07745 [cs.CR] (cit. on p. 17).

[52] Kezhou Ren, Jinshu Zhu, Tao Yuan, Sheng Wen, and Frank Jiang. "ID-RDRL: A Deep Reinforcement Learning-Based Feature Selection Intrusion Detection Model". In: *Scientific Reports* 12 (2022), p. 15370. DOI: 10.1038/s41598-022-19366-3 (cit. on p. 17).

[53] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. "A Survey of Network-Based Intrusion Detection Data Sets". In: *Computers & Security* 86 (2019), pp. 147–167. DOI: 10.1016/j.cose.2019.06.005. arXiv: 1903.02460 (cit. on pp. 3, 7, 8, 12, 23, 25).

[54] Arnaud Rosay, Eloïse Cheval, Florent Carlier, and Pascal Leroux. "Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017". In: *Proceedings of the 8th International Conference on Information Sys-*

tems Security and Privacy (ICISSP). 2022, pp. 25–36. DOI: 10.5220/0010774000003120 (cit. on pp. 10, 20).

[55] Takaya Saito and Marc Rehmsmeier. "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets". In: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432 (cit. on p. 22).

[56] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. "Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection". In: (2021). Proposes standardised NetFlow-aligned feature mapping across CIC and UNSW-NB15 datasets. arXiv: 2104.07183 [cs.CR] (cit. on pp. 6, 13).

[57] Habeeb Mohammed Sayeeduddin and T. Ranga Babu. "Network Intrusion Detection System: A Survey on Artificial Intelligence-Based Techniques". In: *Expert Systems* (2022). DOI: 10.1111/exsy.13066 (cit. on p. 12).

[58] Karen A. Scarfone and Peter M. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Tech. rep. Special Publication 800-94. National Institute of Standards and Technology, 2007. URL: https://csrc.nist.gov/pubs/sp/800/94/final (cit. on pp. 2, 3, 22).

[59] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. "Prioritized Experience Replay". In: *International Conference on Learning Representations*. 2016. arXiv: 1511.05952 (cit. on p. 15).

[60] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "A Detailed Analysis of the CICIDS2017 Data Set". In: *Information Systems Security and Privacy*. Vol. 977. Communications in Computer and Information Science. Springer, 2019, pp. 172–188. DOI: 10.1007/978-3-030-25109-3_9 (cit. on p. 9).

[61] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization". In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116. DOI: 10.5220/0006639801080116. URL: https://www.unb.ca/cic/datasets/ids-2017.html (cit. on pp. 5, 7–9).

[62] Robin Sommer and Vern Paxson. "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection". In: *2010 IEEE Symposium on Security and Privacy (S&P)*. 2010, pp. 305–316. DOI: 10.1109/SP.2010.25 (cit. on pp. 3, 20).

[63] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller. "An Overview of IP Flow-Based Intrusion Detection". In: *IEEE Communications Surveys & Tutorials* 12.3 (2010), pp. 343–356. DOI: 10.1109/SURV.2010.032210.00054 (cit. on p. 5).

[64] Stable-Baselines3 Contributors. *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation; access date 2026. 2023. URL: https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html (cit. on p. 16).

[65] Maxwell Standen, Martin Lucas, David Bowman, Toby J. Richer, Junae Kim, and Damian Marriott. "CybORG: A Gym for the Development of Autonomous Cyber Agents". In: *arXiv preprint arXiv:2108.09118* (2021). Preprint; also cited as an IJCAI-21 Adaptive Cyber Defense workshop paper. arXiv: 2108.09118 [cs.CR] (cit. on p. 17).

[66] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html (cit. on pp. 13, 24).

[67] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. "A Detailed Analysis of the KDD CUP 99 Data Set". In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Appli-*

cations (CISDA). 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528 (cit. on pp. 7, 8).

[68] Ashish Thakkar and Rachna Lohiya. "A Review of the Advancement in Intrusion Detection Datasets". In: *Procedia Computer Science* 167 (2020), pp. 636–645. DOI: 10.1016/j.procs.2020.03.270 (cit. on p. 7).

[69] Muhammad Fahad Umer, Muhammad Sher, and Yaxin Bi. "Flow-Based Intrusion Detection: Techniques and Challenges". In: *Computers & Security* 70 (2017), pp. 238–254. DOI: 10.1016/j.cose.2017.05.009 (cit. on p. 5).

[70] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. "Dueling Network Architectures for Deep Reinforcement Learning". In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. 2016, pp. 1995–2003. URL: https://proceedings.mlr.press/v48/wangf16.html (cit. on p. 15).

[71] Christopher J. C. H. Watkins and Peter Dayan. "Q-Learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292. DOI: 10.1007/BF00992698 (cit. on p. 14).

[72] Jianping Xiao, Chun Long, Jing Zhao, Jinxia Wei, Anlei Hu, and Guanyao Du. "A Survey on Network Intrusion Detection Based on Deep Learning". In: *Frontiers of Data and Computing* 3.3 (2021), pp. 59–74. DOI: 10.11871/jfdc.issn.2096-742X.2021.03.006 (cit. on p. 12).

[73] Wanrong Yang, Alberto Acuto, Yihang Zhou, and Dominik Wojtczak. "A Survey for Deep Reinforcement Learning Based Network Intrusion Detection". In: *Applied AI Letters* 7.2 (2026), e70026. DOI: 10.1002/ail2.70026. arXiv: 2410.07612 [cs.CR] (cit. on pp. 1, 16, 24, 25).